Technical Analysis - Identity Checker

1. Problem Statement

The project addresses identity verification from one image: a selfie where the person is holding a visible identity document. The goal is to verify whether the live face in the scene is compatible with the face printed on the document. This problem is relevant because document-based onboarding workflows often require a user to prove that the document belongs to the person currently present in front of the camera.

The project is an academic prototype. It is not intended for real identity control, legal verification, or production biometric decision-making.

2. Dataset

The local dataset contains 19 single-image samples: 10 real camera photographs and 9 synthetic images. Each image shows a live face and a visible document; labels were assigned manually after comparing the live portrait with the portrait printed on the document. The classes are:

- `match`: the live face and the document face represent the same identity.
- `no_match`: the document face does not match the live face.

There are 12 `match` and 7 `no_match` samples. A fixed, stratified split was created before evaluation: 6 samples (3 per class) in `validation.csv` for threshold selection and 13 samples (9 match, 4 no-match) in `test.csv` for the final metrics. No test image was used to select the threshold. Images containing real identity documents remain local and are excluded by `.gitignore` because they contain biometric and personal data.

The dataset is intentionally small and includes repeated identities. Consequently, the experiment measures performance on these acquisition conditions only and must not be interpreted as population-level biometric validation.

3. Pipeline

The application implements a complete computer vision pipeline:

1. Data acquisition: the Flask interface receives one uploaded image

containing both the live face and the document.
2. Preprocessing: the image is decoded with OpenCV, all visible faces are detected, and face regions are cropped or aligned.
3. Face selection: detected faces are sorted by area. The largest face is treated as the live/selfie face; smaller faces are treated as candidate document portraits.
4. Feature representation: the deep pipeline uses SFace neural embeddings. The fallback pipeline uses grayscale intensity, Local Binary Patterns, and gradient histograms.
5. Core logic: the system compares the live-face embedding with each document-face candidate, keeps the best cosine similarity, and applies a configurable threshold.
6. Post-processing: the best candidate is selected, scores are rounded, and an annotated preview highlights the live face and selected document face.

The project exposes both a browser interface and a JSON API at `/api/verify`, which makes the pipeline usable from scripts or external clients.
It also exposes `/health` for service and model-file status checks during demo or deployment.

The architecture is documented in `docs/architecture.md`, while the API contract is documented in `docs/api.md` and `openapi.json`.

4. Algorithms and Architecture

The primary method uses OpenCV Zoo models:

- YuNet for face detection.
- SFace for face recognition and embedding extraction.

Both models are loaded with OpenCV DNN through `opencv-contrib-python`. The use of pre-trained models is justified because the project focuses on the end-to-end computer vision pipeline and identity-verification workflow, while the custom logic is in the single-image face selection, candidate comparison, thresholding, fallback path, evaluation, and web integration.

Model files can be prepared with `python scripts/download_models.py`. If they are missing at runtime, the app attempts to download them automatically before falling back to the classical pipeline.

The pre-trained model usage, intended scope, limitations, and ethical

constraints are summarized in `docs/model_card.md`. External references are listed in `docs/references.md`.

If the ONNX models are unavailable, the system falls back to a classical method:

- Haar Cascade face detection.
- Histogram equalization.
- Local Binary Pattern texture descriptor.
- Gradient-orientation histogram.
- Cosine similarity and threshold decision.

5. Experimental Protocol

Threshold selection was performed only on the validation split:

```powershell
python scripts/tune_threshold.py validation.csv --output
threshold_tuning.json
```

The threshold grid ranged from 0.20 to 0.80 in steps of 0.02. The selected threshold was `0.32`, based on validation F1-score and accuracy, with the higher threshold used as the final tie-breaker. Final evaluation was then run once on the held-out test set:

```powershell
python evaluate.py test.csv --threshold 0.32 --output
evaluation_results.json --report docs/evaluation_report.md --plots-dir
docs/figures
```

The script reports:

- accuracy;
- precision;
- recall;
- F1-score;
- confusion matrix;
- confusion-matrix plot;
- score-distribution plot;
- row-level predictions and errors.

Metrics are computed at image level. A processing failure is treated as a `no_match` prediction, reflecting the application's fail-closed

behavior.

6. Results

- Threshold: `0.32`
- Samples: `13`
- Positive samples: `9`
- Negative samples: `4`

Metrics

Metric	Value
Accuracy	84.62%
Precision	100.00%
Recall	77.78%
F1-score	87.50%

Confusion Matrix

	Predicted match	Predicted no match
Actual match	7	2
Actual no match	0	4

7. Failure Analysis

Two observed false negatives were:

- `synthetic_005.png`, score `0.274`: the document portrait is small and differs from the live face in scale, expression, and generated facial detail;
- `synthetic_006.png`, score `0.230`: the document portrait occupies very few pixels and differs in hairstyle and capture appearance from the live portrait.

The validation set also exposed two false accepts at the selected threshold (`real_001.png`, score `0.398`, and `synthetic_002.png`, score `0.432`). This instability, combined with the perfect negative-class result on only four test negatives, shows why a larger identity-disjoint dataset is necessary. One validation image (`real_004.jpeg`) contains no portrait on the visible side of the card; the pipeline correctly fails closed because it cannot detect two faces.

Additional known failure modes are:

- the document portrait is too small or blurry;
- glare or plastic reflections hide the document face;
- the live face is partially occluded;
- strong pose differences between live face and document face;
- multiple unrelated faces appear in the scene;
- the largest detected face is not the live user;
- synthetic data is visually too different from real camera images.

The main improvement would be a larger, identity-disjoint dataset with more document types, lighting conditions, poses, occlusions and demographic diversity. Document localization before face detection would also reduce confusion with unrelated background faces.

8. Ethical and Security Considerations

The system processes sensitive biometric and document data. For this reason:

- uploaded files are deleted after verification;
- upload size is limited;
- filenames are sanitized;
- the prototype should not store personal documents or selfies;
- the system should not be used for real identity decisions;
- biometric systems can show demographic bias and different performance across lighting, camera quality, age, skin tone, and document type;
- results should be interpreted as a computer vision experiment, not as proof of identity.

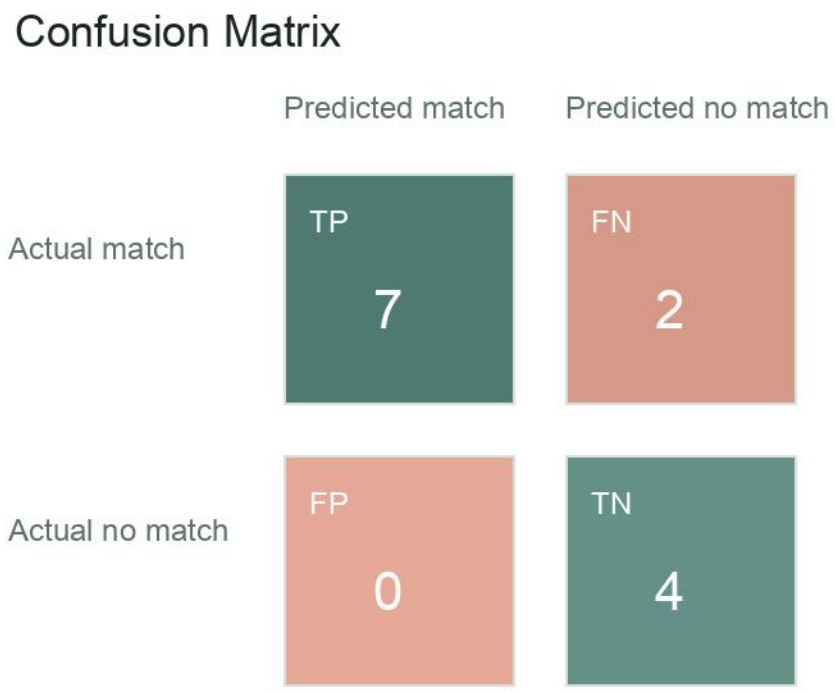
9. Reproducibility

The repository includes:

- Flask source code;
- JSON API endpoint;
- API and architecture documentation;
- OpenAPI specification;
- `requirements.txt`;
- evaluation script;
- generated plots for the experimental section;
- dataset CSV builder and threshold tuning scripts;
- model download/setup script;

- automated tests;
- project audit and submission checklist;
- privacy/ethics documentation;
- model card and references;
- GitHub Actions test workflow;
- Docker deployment files;
- sample CSV format;
- README with setup and running instructions;
- this technical analysis document.

Confusion Matrix



Score Distribution